SymPy Bug Report

1 Bug 13: nonlinsolve returns a solution where the original rational equation is undefined

1.1 Minimal Reproducer

```
import sympy
from sympy import nonlinsolve, symbols

x, y = symbols("x y")
eqs = [(x*y - x)/(x - 1) - y, y - 1]
sol = nonlinsolve(eqs, [x, y])

print("SymPy version:", sympy.__version__)
print("nonlinsolve result =", sol)
for candidate in sol:
    subs = {x: candidate[0], y: candidate[1]}
    print("candidate =", candidate)
    print("residuals =", [eq.subs(subs) for eq in eqs])
print("denominator at (1, 1) =", (x - 1).subs(x, 1))
```

1.2 Observed Behavior

```
SymPy version: 1.14.0
nonlinsolve result = {(1, 1)}
candidate = (1, 1)
residuals = [nan, 0]
denominator at (1, 1) = 0
```

SymPy returns the concrete tuple $(1, 1)$. Direct substitution into the original first equation produces `nan`, because the denominator $x - 1$ vanishes at that point. Thus the returned tuple is not merely an alternate representation of a valid solution; it is outside the domain of the original system.

1.3 Expected Behavior

The mathematically correct solution set is EmptySet. A solver for rational equations must reject every candidate that makes any original denominator zero, even if that candidate solves the denominator-cleared polynomial equations.

1.4 Mathematical Explanation

The system submitted to `nonlinsolve` is

$$\frac{xy - x}{x - 1} - y = 0, \qquad y - 1 = 0.$$

The second equation forces $y = 1$. Clearing the denominator in the first equation gives

$$xy - x - y(x - 1) = 0,$$

which simplifies to $y - x = 0$. Combining this denominator-cleared equation with $y = 1$ forces $x = 1$.

However, clearing the denominator is valid only under the domain condition $x \neq 1$. The only point produced by the cleared polynomial system is exactly the excluded point $(1, 1)$, where the original rational expression has denominator 0. Therefore no valid solution remains, and the correct result is the empty set.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant implementation in `sympy/solvers/solveset.py`, in the public `nonlinsolve` path. The traced call path is

$$\texttt{nonlinsolve} \to \texttt{\_separate\_poly\_nonpoly} \to \texttt{\_simple\_dens} \to \texttt{\_handle\_poly},$$

followed by an early return in the zero-dimensional polynomial success path.

The helper `_separate_poly_nonpoly` records denominators before converting rational equations to numerators. For the reproducer, it records the denominator $x-1$, then replaces the first rational equation by its numerator, which reduces to $-x + y$. The polynomial subsystem $[-x + y,\ y - 1]$ is then solved as $(x, y) = (1, 1)$.

The diagnosed bug is that when this polynomial subsystem solves all equations, `nonlinsolve` returns the polynomial solutions directly instead of applying the recorded denominator exclusions. The denominator-aware `substitution(..., exclude=denominators)` path is used only later, when remaining equations still need processing.

```
if not remaining:
    # If there is nothing left to solve then return the solution from
    # solve_poly_system directly.
    return FiniteSet(*map(to_tuple, poly_sol))
```

This control-flow choice causes the observed mathematical failure: the implementation has already recorded that $x - 1$ must be nonzero, but the early return bypasses that exclusion and admits the exact point where the denominator vanishes. A plausible fix direction supported by the diagnosis is to filter `poly_sol` against the recorded denominators before this early return, or to route the fully solved rational-polynomial case through the same denominator-aware validation used by `substitution`.

1.6 Additional Failing Instances

The independent verification checks the representative case and five shifted instances of the same family:

$$\left[\frac{xy - ax}{x - a} - y,\ y - a\right], \qquad a \in \{1, 2, 3, 4, 5, 6\}.$$

For each a, the second equation forces $y = a$. The denominator-cleared first equation simplifies to $ay - ax = 0$, so the cleared system forces $x = a$. That is exactly the excluded value of the denominator $x - a$, so the expected result is empty throughout the family.

Input / Expression	**SymPy behavior**	**Expected behavior**
$a = 1:\ \frac{xy-x}{x-1} - y = 0,\ y = 1$	Returns $\{(1, 1)\}$; $x - 1 = 0$.	EmptySet.
$a = 2:\ \frac{xy-2x}{x-2} - y = 0,\ y = 2$	Returns $\{(2, 2)\}$; $x - 2 = 0$.	EmptySet.
$a = 3:\ \frac{xy-3x}{x-3} - y = 0,\ y = 3$	Returns $\{(3, 3)\}$; $x - 3 = 0$.	EmptySet.
$a = 4:\ \frac{xy-4x}{x-4} - y = 0,\ y = 4$	Returns $\{(4, 4)\}$; $x - 4 = 0$.	EmptySet.
$a = 5:\ \frac{xy-5x}{x-5} - y = 0,\ y = 5$	Returns $\{(5, 5)\}$; $x - 5 = 0$.	EmptySet.
$a = 6:\ \frac{xy-6x}{x-6} - y = 0,\ y = 6$	Returns $\{(6, 6)\}$; $x - 6 = 0$.	EmptySet.

1.7 Related Issues Assessment

The best-effort deduplication check found general background on extraneous solutions introduced by clearing denominators and identified the relevant public SymPy source context, but it did not find a public SymPy issue, pull request, Stack Overflow post, mailing-list discussion, or release note containing this exact system, the returned result $\{(1, 1)\}$, the residuals $[\texttt{nan}, 0]$, or the diagnosed bypass of `exclude=denominators`. The deduplication verdict is therefore a new failure mode with no public precedent. The bug remains individually actionable because it supplies a minimal reproducible system, a concrete invalid solution, a source-level control-flow explanation, and a concise regression test.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import EmptySet, nonlinsolve, symbols


@pytest.mark.parametrize("a", [1, 2, 3, 4, 5, 6])
def test_nonlinsolve_rejects_zero_denominator_solution(a):
    x, y = symbols("x y")
    eqs = [(x*y - a*x)/(x - a) - y, y - a]
    assert nonlinsolve(eqs, [x, y]) == EmptySet
```